Linux File Permissions Security Audit

Project description

As a security professional working with the organization's research team, ensuring data integrity and system security through proper access control is vital. In this project, I examined the existing file system permissions within the projects directory to identify instances of non-compliance with organizational security policies. Using Linux commands, I audited user, group, and other permissions, and subsequently modified them to enforce least-privilege access, restrict unauthorized write capabilities, and secure sensitive research directories.

Check file and directory details

To audit the file system and view comprehensive permission settings, ownership, and hidden files within the projects directory, the following command is used:

```
ls -la
```

- Command Explanation:

ls: Lists directory contents.

-l: Uses the long listing format, which displays file type, permissions, ownership, size, and modification timestamps.

-a: Includes hidden files (those starting with a dot) in the listing layout.

File System Directory Output: The command output provides an explicit 10-character permission matrix for each file asset. The baseline state of the target environment reports as follows:

drwxr-xr-x researcher2 research_team . (Current Directory)
drwxr-xr-x researcher2 research_team .. (Parent Directory)
drwxr-xr-x researcher2 research_team drafts
-rw-rw-rw- researcher2 research_team project_k.txt
-r--r--r-- researcher2 research_team .project_x.txt

Describe the permissions string

Let us break down and analyze the foundational structure of a 10-character permissions string, utilizing project_k.txt (-rw-rw-rw-) as our active demonstration baseline.

- Component Breakdown:

Position 1 (File Type Indicator): The leading character '-' denotes a standard regular file. (A 'd' represents a directory structural container).

Positions 2–4 (User / Owner Domain): 'rw-' indicates that the primary owner (researcher2) holds exclusive Read (r) and Write (w) permissions, but lacks Execute (-) flags.

Positions 5–7 (Group Domain): 'rw-' confirms that assigned workspace security group members (research_team) have active Read (r) and Write (w) rights, without Execute access.

Positions 8–10 (Other / Global Domain): 'rw-' means all global system platform profiles hold full default Read (r) and Write (w) access flags.

Change file permissions

The Operational Vulnerability: The initial configurations for project_k.txt (-rw-rw-rw-) show that both the group and global accounts maintain active workspace write capabilities.

To immediately mitigate the risk vector and align permissions with corporate requirements, the following operational administrative command execution is performed:

```
chmod go-w project_k.txt
```

- Command Action Analysis:

chmod: Fires the core Linux binary module dedicated to manipulating object mode security bits.

go-w: Targets Group (g) and Others (o), implementing a subtraction operator (-) explicitly targeting the Write (w) performance capability flag.

project_k.txt: Identifies the absolute target system file constraint context.

Post-remediation string validation: -rw-r----

Change file permissions on a hidden file

The Operational Vulnerability: The hidden archive .project_x.txt exhibits global open read-only states (-r--r--r--). Operational guidelines mandate that while the internal security user and corresponding team group require continuous visibility access, external system profiles must be cleanly locked out.

To remove default read exposure pathways from unauthorized global profiles, use the following syntax:

```
chmod o-r .project_x.txt
```

Alternative representation utilizing absolute octal configuration values: chmod 440 .project_x.txt

- Command Action Analysis:

o-r: Targets the Others (o) bit string boundary, parsing a deletion instruction explicitly targeting Read (r) authorization flags.

.project_x.txt: Specifies the hidden research volume matrix component.

Post-remediation string validation: -r--r-----

Change directory permissions

The Operational Vulnerability: The drafts directory contains unvetted corporate intellectual property. Organization security principles state that access to this target path must be isolated entirely to the asset owner (researcher2), revoking access from all neighboring user and group frames.

To enforce an isolated silo boundary layer over the dynamic path layout, execute:

```
chmod 700 drafts
```

- Command Action Analysis:

700: Absolute octal evaluation parameter matrix where:
- 7 (4+2+1) provides the master explicit owner full dynamic Read (r), Write (w), and Directory Traversal Execution (x) capacity.
- 0 strips the local working structural Group domain of all accessible bit parameters (---).
- 0 drops the global Others identity tracking completely from the tracking boundary (---).

Post-remediation string validation: drwx------

Summary

Through this targeted security remediation audit, I successfully locked down the research team's project environment to adhere strictly to the principle of least privilege. By leveraging system commands such as ls -la and chmod, I audited permission vulnerabilities, eliminated non-compliant public write permissions on active project files, and isolated sensitive archive configurations. Finally, by restricting the drafts directory exclusively to its owner, I mitigated risk vectors surrounding pre-publication research data, ensuring a hardened and compliant local storage infrastructure.